Estructuras de Datos

2026
Segundo Cuatrimestre

REPASO

Repaso

Estructuras como Tipos de Datos Abstractos

TDA Pila

TDA Cola

TDA Lista con Posición

Repaso

Estructuras como Tipos de Datos Abstractos

TDA Pila

TDA Cola

TDA Lista con Posición

Opciones para implementar las estructuras

TDA Pila

TDA Cola

TDA Lista con Posición

Nodo Enlazado

Nodo Doblemente Enlazado

Arrays, y otras clases provistas por Java

Recorrido manual vs Iteradores



(lista no vacía

...

```
Position<E> p = lista.first();

Position<E> fin = lista.last();

while(p != fin) {

    E element = p.element();

    p = lista.next(p);

}

E element = p.element();
```

```
Iterator<Integer> iterador = numeros.iterator();

while (iterador.hasNext()) {
    Integer numero = iterador.next();
    System.out.println(numero);
}

for (Integer numero: numeros) {
    System.out.println(numero);
}
}
```

Recorrido manual2 vs Iteradores

(lista no vacía)

```
...
Position<E>  p  =  lista.first()
Position<E>  fin  =  lista.last()
while(p  !=  null)
    E element = p.element();

    p = p == fin() ? null: lista.next(p),

}
```

~~E element = p.element() ;~~

```
Iterator<Integer> iterador = numeros.iterator();

while (iterador.hasNext()) {
    Integer numero = iterador.next();
    System.out.println(numero);
}

for (Integer numero: numeros) {
    System.out.println(numero);
}
}
```

Recorrido manual3 (adentro del TDA) vs Iteradores

```
DNode<E> pos = this.header.getNext();
DNode<E> fin = this.trailer;
while(pos != fin)
    E element = p.element();

    p = pos.getNext();

}
```

~~E element = p.element() ;~~

```
Iterator<Integer> iterador = numeros.iterator();

while (iterador.hasNext()) {
    Integer numero = iterador.next();
  { System.out.println(numero);
}

for (Integer numero: numeros) {
    System.out.println(numero);
}
}
```

TDA MAPEO


AFA
FIFA WORLD CUP
Qatar2022
FRANCE
FFF
23 E. MARTÍNEZ
26 MOLINA
13 ROMERO
19 OTAMENDI
3 TAGLIAFICO
7 DE PAUL
24 FERNÁNDEZ
20 MAC ALLISTER
10 MESSI (C)
9 ÁLVAREZ
11 DI MARÍA
SUPLENTES
1. ARMANI, 12.RULLI, 2. FOYTH, 4. MONTIEL,
5. PAREDES, 6. PEZZELLA, 8. ACUÑA,
14. PALACIOS, 15. CORREA, 16. ALMADA,
17. GÓMEZ, 18. RODRÍGUEZ, 21. DYBALA,
22. L. MARTÍNEZ, 25. LI. MARTÍNEZ
DT: LIONEL SCALONI
10
86
78
PedidosYa
BINANCE
Schneider
Aerolíneas Argentinas
BETWARRIOR
KAVAK
Duelbits
KONAMI

Legajo	Estado
137843	Aceptada
137344	Aceptada
133892	Aceptada
141035	Aceptada
126765	Aceptada
137343	Aceptada
134667	Aceptada
139472	Aceptada
140389	Aceptada
142060	Aceptada
135641	Aceptada
129379	Aceptada
137379	Aceptada
142101	Aceptada
130048	Aceptada
132508	Aceptada
55615	Aceptada
126035	Aceptada
134726	Aceptada
118582	Aceptada
130953	Aceptada
137841	Aceptada
122040	Pendiente
140981	Aceptada
142080	Aceptada
122659	Aceptada
120088	Aceptada
133427	Aceptada
140350	Aceptada
142046	Aceptada
122551	Aceptada
137800	Aceptada
140691	Aceptada
126895	Aceptada
119479	Aceptada
140983	Aceptada
142106	Aceptada
138731	Pendiente



™ & © 2020 SCG POWER RANGERS LLC AND HASBRO. ALL RIGHTS RESERVED.

In development, not final. Final product may vary from prototype images.

TDA Mapeo

Los mapeos proporcionan una forma de almacenar y recuperar datos basados en un identificador único (**la clave**).

TDA Mapeo

Los mapeos proporcionan una forma de almacenar y recuperar datos basados en un identificador único (**la clave**).

Clave: Atributo de un individuo que sirve para identificarlo en un conjunto de individuos

TDA Mapeo

Ejemplos de Claves son:

DNI, Número de libreta para alumnos

Número de afiliado para afiliados a una obra social

Número de cuenta para una cuenta de un banco

Color de un Power Ranger

Patente (Letras y Números) para un vehículo

TDA Mapeo - Interface

Como parte de la interface de Mapeo, vamos a introducir el concepto de entrada o Entry<K,V> que vendría a representar el par (K, V).

TDA Mapeo - Interface

Como parte de la interface de Mapeo, vamos a introducir el concepto de entrada o Entry<K,V> que vendría a representar el par (K, V).

```
public interface Entry<K,V> {
    public K getKey(); // Retorna la clave de la entrada
    public V getValue(); // Retorna el valor de la entrada
}
```

TDA Mapeo - Interface

```
public class Entrada<K,V> implements Entry<K,V> {
        private K clave;
        private V valor;

        // Constructor
        public Entrada(K clave, V valor) { this.clave = clave; this.valor = valor; }


        public K getKey() { ... } // Getters
        public V getValue() { ... }

        public void setKey( K clave ) { ... } // Setters
        public void setValue(V valor) { ... }

        public String toString( ) { // Para mostrar entradas
                return "(" + getKey() + "," + getValue() + ")" ; }

}
```

TDA Mapeo - Interface

```
public interface Map<K,V> { //K representa el elemento clave y V el elemento asociado a la clave
        public int size();
        public boolean isEmpty();
        public V get(K key) throws InvalidKeyException;
        public V put(K key, V value) throws InvalidKeyException;
        public V remove(K key) throws InvalidKeyException;
        public Iterable<K> keys();
        public Iterable<V> values();
        public Iterable<Entry<K,V>> entries();
}
```

TDA Mapeo - Interface

size(): Retorna el número de entradas de M

isEmpty(): Testea si M es vacío

get(k): Si M contiene una entrada e con clave igual a k, retorna el valor de v; sino retorna null

put(k,v): Si M no tiene una entrada con clave k, entonces agrega una entrada (k,v) a M y retorna null. Si M ya tiene una entrada e con clave k, reemplaza el valor con v en e y retorna el valor viejo de e.

TDA Mapeo - Interface

remove(k): Remueve de M la entrada con clave k y retorna su valor. Si M no tiene entrada con clave k, retorna null

keys(): Retorna una colección iterable de las claves en M.

keys().iterator(): Retorna un iterador de claves.

values(): Retorna una colección iterable con los valores de las claves almacenadas en M

values().iterator(): Retorna un iterador de valores

entries(): Retorna una colección iterable con las entradas de M

entries().iterator(): Retorna un iterador de entradas

Ejem

tiempo	Operación	Salida	Mapeo M de enteros en caracteres
1	isEmpty()	true	∅
2	put(5, A)	null	{(5,A)}
3	put(7,B)	null	{(5,A), (7,B)}
4	put(2,C)	null	{(5,A), (7,B), (2,C)}
5	put(8,D)	null	{(5,A), (7,B), (2,C), (8,D)}
6	put(2,E)	C	{(5,A), (7,B), (2,E), (8,D)}
7	get(7)	B	{(5,A), (7,B), (2,E), (8,D)}
8	get(4)	null	{(5,A), (7,B), (2,E), (8,D)}
9	get(2)	E	{(5,A), (7,B), (2,E), (8,D)}
10	size()	4	{(5,A), (7,B), (2,E), (8,D)}
11	remove(5)	A	{(7,B), (2,E), (8,D)}
12	remove(2)	E	{(7,B), (8,D)}
13	get(2)	null	{(7,B), (8,D)}
14	isEmpty()	false	{(7,B), (8,D)}

TDA Mapeo

Los mapeos se pueden implementar con:
-Listas
-Árboles de búsqueda (parte final de la materia)
-Hashing (próxima clase)

TDA Mapeo

Implementemos el TDA Mapeo utilizando una lista.

Estructura de datos y constructor

Implementamos el mapeo con una clase genérica MapeoConLista

```
public class MapeoConLista<K, V> implements Map<K, V> {

  protected PositionList<Entrada<K,V>> S;

  public MapeoConLista() {

        S = new ListaDoblementeEnlazada<Entrada<K,V>>();

  }

...
```

```
public interface Map<K,V> { //K representa el elemento clave y V el elemento asociado a la clave

    public int size();

    public boolean isEmpty();

    public V get(K key) throws InvalidKeyException;

    public V put(K key, V value) throws InvalidKeyException;

    public V remove(K key) throws InvalidKeyException;

    public Iterable<K> keys();

    public Iterable<V> values();

    public Iterable<Entry<K,V>> entries();

}
```

Implementación de mapeo con TDAlista: put

put(k,v): Si M no tiene una entrada con clave k, entonces agrega una entrada (k,v) a M y retorna null. Si M ya tiene una entrada e con clave k, reemplaza el valor con v en e y retorna el valor viejo de e.

```
public V put( K key, V value ) {
   // Para cada posición p de la lista S hacer:

     // Si la clave de la entrada en la posición p es key:

       // Salvar el valor de la entrada en aux

       // Setear el nuevo valor de la entrada a value

       // Retornar el viejo valor

     }
  } // Si salí del for-each entonces no encontré una entrada con clave key
                                                    // Inserto una nueva entrada (key,value)
                  // Retorno null para indicar que inserté una nueva entrada
}
```

Implementación de mapeo con TDAlista: put

put(k,v): Si M no tiene una entrada con clave k, entonces agrega una entrada (k,v) a M y retorna null. Si M ya tiene una entrada e con clave k, reemplaza el valor con v en e y retorna el valor viejo de e.

```
public V put( K key, V value ) {
  checkKey(key);
   // Para cada posición p de la lista S hacer:
  for( Position<Entrada<K,V>> p : S.positions() ) {
    // Si la clave de la entrada en la posición p es key:
    if( p.element().getKey().equals( key ) {
      // Salvar el valor de la entrada en aux
      V aux = p.element().getValue();
      // Setear el nuevo valor de la entrada a value
      p.element().setValue( value );
      // Retornar el viejo valor
      return aux;
    }
  } // Si salí del for-each entonces no encontré una entrada con clave key
  S.addLast(new Entrada<K,V>(key, value) ); // Inserto una nueva entrada (key,value)
  return null; // Retorno null para indicar que inserté una nueva entrada
}
```

Implementación de mapeo con TDAlista: put (return al final)

put(k,v): Si M no tiene una entrada con clave k, entonces agrega una entrada (k,v) a M y retorna null. Si M ya tiene una entrada e con clave k, reemplaza el valor con v en e y retorna el valor viejo de e.

```
public V put( K key, V value ) {
  checkKey(key);
   // Para cada posición p de la lista S hacer:
  Iterator<Position<Entrada<K,V>> it = S.positions().iterator();
  V resultado = null;
  while( it.hasNext() && (resultado == null)) {
    // Si la clave de la entrada en la posición p es key:
    Position<Entrada<K,V>> p = it.next();
    if( p.element().getKey().equals( key ) {
      // Salvar el valor de la entrada en resultado
      resultado = p.element().getValue();
      // Setear el nuevo valor de la entrada a value
      p.element().setValue( value );
    }
  } // Si salí del for-each entonces no encontré una entrada con clave key
  if (resultado==null) S.addLast(new Entrada<K,V>(key, value) ); // Inserto una nueva entrada (key,value)
  return resultado; // Retorno null para indicar que inserté una nueva entrada
}
```

Implementación de mapeo con TDAlista: get

get(k): Si M contiene una entrada e con clave igual a k, retorna el valor de v; sino retorna null

```
public V get(K key) {
  checkKey(key);
  // Para cada posición p de la lista S hacer:
  for( Position<Entrada<K,V>> p : S.positions() ) {
    // Si la clave de la entrada actual es key:
    if( p.element().getKey().equals( key ) ) {
      // Retornar el valor de la entrada actual:
      return p.element().getValue();
    }
  }
  // Si salí del for-each, quiere decir que no encontré ninguna entrada con clave key.
  return null;
}
```

Tarea: reimplementar con un solo return al final.

Implementación de mapeo con TDAlista: remove

remove(k): Remueve de M la entrada con clave k y retorna su valor. Si M no tiene entrada con clave k, retorna null

```
public V remove(K key ) {
  // Para cada posición p de S hacer:
  for( Position<Entrada<K,V>> p : S.positions() )
    // Si la entrada de la posición p tiene clave key:
    if( p.element().getKey().equals( key ) ) {
      // Salvar el valor de la entrada corriente en value:
      V value = p.element().getValue();
      // Eliminar la posición p de la lista S:
      S.remove( p );
      // Retornar el valor de entrada removida del mapeo:
      return value;
}
// Si salí del for-each, quiere decir que no encontré ninguna entrada con clave key
return null;
}
```

Notar que se hace un único cambio pero es un iterador de posiciones

Reimplementar con un solo return al final

Aplicación

Problema: Dada una cola Q de caracteres, determinar la frecuencia de apariciones de cada carácter.

- La frecuencia de apariciones de un carácter C en una sequencia de caracteres Q se define como la cantidad de apariciones de C dividido la cantidad total de caracteres de Q.
- Ejemplo: Si ‘R’ apareció 4 veces y hay 200 caracteres en la secuencia Q, entonces la frecuencia de apariciones de ‘R’ en Q es: 4 / 200 = 0,02 = 2%

```
public static Map<Character, Float> calcularFrecuenciaApariciones(Queue<Character> q ) {

  Map<Character, Integer> m = new MapeoConLista<Character, Integer>();
  int cantidadTotalCaracteres = q.size(); // Salvo el largo de q
  while( !q.isEmpty() ) { // mientras la cola q no se vacíe
    char c = q.dequeue(); // obtener un elemento c de la cola q
    if ( m.get(c) == null ) // Si es la primera vez que veo a c
      m.put( c, 1 ); // Anoto que c apareció exactamente 1 vez
    else
      m.put( c, m.get(c) + 1 ); // Anoto que c apareció una vez más
   }




}
```

```
public static Map<Character, Float> calcularFrecuenciaApariciones(Queue<Character> q ) {

  Map<Character, Integer> m = new MapeoConLista<Character, Integer>();
  int cantidadTotalCaracteres = q.size(); // Salvo el largo de q
  while( !q.isEmpty() ) { // mientras la cola q no se vacíe
    char c = q.dequeue(); // obtener un elemento c de la cola q
    if ( m.get(c) == null ) // Si es la primera vez que veo a c
      m.put( c, 1 ); // Anoto que c apareció exactamente 1 vez
    else
      m.put( c, m.get(c) + 1 ); // Anoto que c apareció una vez más
   }
  // Armo un mapeo nuevo para retornar las frecuencias:
  Map<Character, Float> resultado = new MapeoConLista<Character, Float>();
  // Para cada entrada e de m hacer:
  for( Entry<Character, Integer> e : m.entries() )
    // La frecuencia de e.getKey es e.getValue / largo original de q
    // Casteo entero a real para forzar a que / se interprete como división real:
    resultado.put(e.getKey(), ((float) e.getValue()) / cantidadTotalCaracteres );
  return resultado; // Retorno el mapeo resultado
}
```

Break

TDA Mapeo

Como tuvimos en cuenta en la implementación, una condición fundamental del Mapeo es que no permite tener dos entradas con la misma clave.

TDA Mapeo

Una condición fundamental del Mapeo es que no permite tener dos entradas con la misma clave.

put(k,v): Si M no tiene una entrada con clave k, entonces agrega una entrada (k,v) a M y retorna null. Si M ya tiene una entrada e con clave k, reemplaza el valor con v en e y retorna el valor viejo de e.

TDA Mapeo

Una condición fundamental del Mapeo es que no permite tener dos entradas con la misma clave.

put(k,v): Si M no tiene una entrada con clave k, entonces agrega una entrada (k,v) a M y retorna null. Si M ya tiene una entrada e con clave k, **reemplaza el valor con v en e y retorna el valor viejo de e**.

TDA Mapeo

Bajo ciertas condiciones, podía ser necesario permitir que una clave tenga asociada múltiples valores.

Diccionario esencial de la lengua española

Escriba aquí la palabra

áéíóúüñ

Debe tenerse en cuenta que esta versión electrónica da acceso al texto del *Diccionario esencial de la lengua española*, publicado en 2006.

Ver definición de *casa* en la edición actual del *Diccionario de la lengua española*

casa. F. **1.** Edificio para habitar. *Una casa de ocho plantas.* ‖ **2.** Edificio de una o pocas plantas destinado a vivienda unifamiliar, en oposición a *piso. Quieren vender el piso y comprarse una casa.* ‖ **3. piso** (‖ vivienda). *Mi casa está en el 3.º C.* ‖ **4.** Edificio, mobiliario, régimen de vida, etc., de alguien. *Echo de menos las comodidades de casa.* ‖ **5. familia** (‖ grupo de personas que viven juntas). *La administración de cada casa.* ‖ **6.** Descendencia o linaje que tiene un mismo apellido y viene del mismo origen. *Casa de los Ríos.* ‖ **7.** Establecimiento industrial o mercantil. *Esta casa es la más antigua en su ramo.* ‖ **8.** Institución de carácter sociocultural y recreativo que agrupa a personas con vínculos geográficos o intereses comunes. *Casa de Galicia. Casa de la cultura.* ‖ **9.** Sede de esta institución. ‖ **10.** hist. Conjunto de estados, vasallos y rentas que poseía un señor. ‖ **11.** pl. *Chile.* **casa** principal de un fundo. ‖ **~ abierta.** F. Domicilio, estudio o despacho de quien ejerce una profesión, un arte o una industria. ‖ **~ civil.** F. En España, conjunto de personas que tenían a su cargo los servicios no militares del palacio o residencia del jefe del Estado. ‖ **~ consistorial.** F. **casa** de la villa o ciudad adonde concurren los concejales de su ayuntamiento a celebrar sus juntas. ‖ **~ cuartel.** F. Puesto o destacamento permanente de un cuerpo armado dotado de viviendas destinadas a los individuos de este cuerpo y sus familias. ‖ **~ cuna.** F. **inclusa.** ‖ **~ cural.** F. **casa parroquial.** ‖ **~ de altos.** F. *Am.* Vivienda que tiene dos o más pisos sobre la planta baja. ‖ **~ de baños.** F. Establecimiento en que se tienen baños para el servicio público. ‖ **~ de beneficencia.** F. Hospital, hospicio o asilo. ‖ **~ de campo.** F. La que está fuera de poblado y sirve para cuidar del cultivo, para recrearse o para ambos objetos a la vez. ‖ **~ de citas.** F. Aquella en que se facilita, clandestinamente, y a cambio de dinero, habitación para las relaciones sexuales. ‖ **~ de comidas.** F. **figón.** ‖ **~ de Dios.** F. Templo o iglesia. ‖ **~ de empeño,** o **~ de empeños.** F. Establecimiento donde se presta dinero mediante la entrega condicionada de alhajas o ropas u otros bienes muebles, en prenda. ‖ **~ de huéspedes.** F. Aquella en que, mediante cierto precio, se da estancia y comida, o solo alojamiento, a algunas personas. ‖ **~ de juego.** F. La que está destinada a la explotación de juegos de azar. ‖ **~ de labor,** o **~ de labranza.** F. Aquella en que habitan los labradores y en la que tienen sus ganados y aperos. ‖ **~ de lenocinio.** F. La de prostitutas. ‖ **~ de locos.** F. **1. manicomio.** ‖ **2.** coloq. Lugar donde hay mucho bullicio, inquietud y falta de autoridad. ‖ **~ del rey.** F. **1. casa real.** ‖ **2.** En España, conjunto de personas civiles y militares que asisten al Rey en el desempeño de sus funciones. ‖ **~ del Señor.** F. **casa de Dios.** ‖ **~ de malicia.** F. hist. Cada una de las que, con cabida para alojar dos familias, se edificaban en Madrid durante el siglo XVII, sobre traza arquitectónica en fachada de un solo piso, rehuyendo así la carga de aposento. ‖ **~ de mancebía.** F. **casa de lenocinio.** ‖ **~ de moneda.** F. La destinada a fundir, fabricar y acuñar moneda. ‖ **~ de oración.** F. **casa de Dios.** ‖ **~ de postas.** F. hist. Parada donde tomaban caballos de refresco los correos y quienes viajaban en posta. ‖ **~ de préstamos.** F. **casa de empeño.** ‖ **~ de prostitución.** F. **casa de lenocinio.** ‖ **~ de pupilos.** F. **casa de huéspedes.** ‖ **~ de putas.** F. **1.** coloq. **casa de lenocinio.** ‖ **2.** coloq. Lugar de gran desorden. ‖ **~ de recreo.** F. La situada en el campo como lugar de descanso y distracción. ‖ **~ de socorro.** F. Establecimiento benéfico donde se prestan los primeros auxilios facultativos a heridos o víctimas de una enfermedad repentina. ‖ **~ de tócame Roque.** F. coloq. Aquella en que vive mucha gente y hay mala dirección y el consiguiente desorden. ‖ **~ de tolerancia.** F. **casa de lenocinio.** ‖ **~ de trato.** F. **casa de lenocinio.** ‖ **~ de vecindad.** F. La que contiene muchas viviendas reducidas, por lo común con acceso a patios y corredores. ‖ **~ dezmera,** o **~ excusada.** F. hist. La principal de cada parroquia que, por privilegio pontificio, pagaba sus diezmos a la real Hacienda en vez de pagarlos a la Iglesia. ‖ **~ fuerte.** F. **1.** La fabricada para habitar en ella, con medios para defenderse de los enemigos. ‖ **2.** La muy acaudalada. ‖ **~ grande.** F. **casa** aristocrática o importante por su riqueza. ‖ **~ militar.** F. En España, conjunto de militares que se hallaban como ayudantes al servicio inmediato del jefe del Estado. ‖ **~ mortuoria.** F. **casa** donde recientemente ha muerto alguna persona. ‖ **~ parroquial.** F. La destinada a vivienda del párroco y que, generalmente, es propiedad de la Iglesia. ‖ **~ paterna.** F. **casa** de los padres. ‖ **~ pública.** F. **casa de lenocinio.** ‖ **~ real.** F. **1. palacio** (‖ **casa** de los reyes). ‖ **2.** Personas reales y conjunto de sus familias. ‖ **~ rectoral.** F. **casa parroquial.** ‖ **~ rodante.** F. *Am.*

TDA DICCIONARIO

TDA Diccionario

Un Diccionario maneja entradas del tipo <K, V> igual que el Mapeo, pero a diferencia de este permite múltiples entradas con la misma clave.

TDA Diccionario - Interfaz

```
public interface Dictionary<K,V> {
        public int size();
        public boolean isEmpty();
        public Entry<K,V> find(K key);
        public Iterable<Entry<K,V>> findAll(K key);
        public Entry<K,V> insert(K key, V value);
        public Entry<K,V> remove( Entry<K,V> e ) throws NonExistentEntryException;
        public Iterable<Entry<K,V>> entries();
}
```

TDA Diccionario - Interfaz

size(); // Retorna el número de entradas de D

isEmpty(); // Testea si D está vacío

find(K key); // Si D contiene una entrada e con clave igual a key, entonces retorna e, sino retorna null

findAll(K key); // Retorna una colección iterable conteniendo todas las entradas con clave igual a key

insert(K key, V value); // Inserta en D una entrada e con clave key y valor value y retorna la entrada e

remove(Entry<K,V> e) throws NonExistentEntryException; //Remueve de D una entrada e, retornando la entrada removida; ocurre un error si e no está en D

entries(); // Retorna una colección iterable con las entradas clave-valor de D.

Ejemplo

Tiempo	Operación	Salida	Diccionario de enteros en caracteres
1	insert(5,A)	(5,A)	{(5,A)}
2	insert(7,B)	(7,B)	{(5,A), (7,B)}
3	insert(2,C)	(2,C)	{(5,A), (7,B), (2,C)}
4	insert(8,D)	(8,D)	{(5,A), (7,B), (2,C), (8,D)}
5	insert(2,E)	(2,E)	{(5,A), (7,B), (2,C), (8,D), (2,E)}
6	find(7)	(7,B)	{(5,A), (7,B), (2,C), (8,D), (2,E)}
7	find(4)	null	{(5,A), (7,B), (2,C), (8,D), (2,E)}
8	find(2)	(2,C)	{(5,A), (7,B), (2,C), (8,D), (2,E)}
9	findAll(2)	{(2,C), (2,E)}	{(5,A), (7,B), (2,C), (8,D), (2,E)}
10	size()	5	{(5,A), (7,B), (2,C), (8,D), (2,E)}
11	remove(find(5))	(5,A)	{(7,B), (2,C), (8,D), (2,E)}
12	find(5)	null	{(7,B), (2,C), (8,D), (2,E)}

TDA Diccionario

Las opciones de implementación son las mismas que las mencionadas para el TDAMapeo.

Ejercicio

Implementar el TDA Diccionario con una lista.

Implementación de diccionarios con Listas

```
public class DiccionarioConLista<K,V> implements Dictionary<K,V> {
  protected ListaDoblementeEnlazada<Entry<K,V>> D;

  public DiccionarioConLista() { D = new ListaDoblementeEnlazada<Entry<K,V>>(); }
  public Iterable<Entry<K,V>> entries() { return D; }

  // Agrega una entrada (key,value) a D y la devuelve
  public Entry<K,V> insert(K key, V value ) {
    Entry<K,V> e = new Entrada<K,V>(key, value);
    D.addLast( e );
    return e;
  }
  // remove(e): Remueve la entrada e del diccionario y retorna la entrada removida.
  public Entry<K,V> remove( Entry<K,V> e ) throws NonExistentEntryException {
    for( Position<Entry<K,V>> p : D.positions() ) {
      if( p.element() == e ) { D.remove(p); return e; }
    }
    throw new NonExistentEntryException( "Diccionario::remove: ..." );
}
```

```
public Iterable<Entry<K,V>> findAll( K key ) {
   PositionList<Entry<K,V>> l = new ListaDoblementeEnlazada<Entry<K,V>>();
   for (Position<Entry<K,V>> p : D.positions() )
      if (p.element().getKey().equals(key) )
         l.addLast( p.element() );
   return l;
}
```

Al usar positions() hay una pasada inicial sobre la lista D.

Se puede evitar usando las posiciones directamente con first, last y next.

TDA CONJUNTO

TDA Conjunto

Un conjunto se define como una estructura de datos que almacena una colección de elementos distintos. Es una estructura de datos fundamental que se utiliza para almacenar y manipular un grupo de objetos, donde cada objeto es único. El conjunto no permite elementos duplicados.

TDA Conjunto

Un conjunto, caracterizado con el tipo Set<E>, tiene definidas las operaciones:

insert(x): Inserta un elemento x el conjunto

delete(x): Elimina el elemento x del conjunto

member(x): Retorna true si x pertenece al conjunto

intersection(S): Intersecta al conjunto con otro conjunto S y retorna un nuevo conjunto intersección

union(S): Une al conjunto con el conjunto S y retorna un nuevo conjunto unión

complement(): Calcula y retorna el complemento del conjunto (solo se puede implementar si el dominio es finito)

iterator(): Retorna un iterador con los elementos del conjunto

Conjuntos: implementación

Usando una lista doblemente enlazada:

- insert(x): insertar x al principio de la lista
- delete(x): Buscar x en la lista y eliminarlo
- member(x): Buscar x en la lista
- intersection(S): por cada elemento x de this buscarlo en S y si está en S insertarlo en el resultado.
- union(S): cada elemento de this se inserta en el resultado y cada elemento de S si no está en el resultado se inserta en el resultado.
- complement(): para cada elemento del dominio si no está en this se agrega al resultado.
- iterator(): Retornar un iterador para la lista subyacente

Tarea: Implementar la interfaz Set<E> usando como representación interna: (1) una lista enlazada de E's y (2) un mapeo de E en Boolean

Bibliografía

Capítulo 9 de M. Goodrich & R. Tamassia, Data Structures and Algorithms in Java. Fourth Edition, John Wiley & Sons, 2006

THE END.